



A Computational Investigation into Rocket Flight

Evaluating Trajectory Through Numerical
Integration, Differentiation, and Root-Finding

Alex Korte

Julia Jaques

Supervisor: Professor Dong Lin, Ph.D.
*Department of Physics and Astronomy, Clemson
University*

College of Science
Department of Physics and Astronomy
Physics 3500

Final Report

Clemson, South Carolina
December 2025

Contents

List of Figures ii

Glossary ii

Symbols ii

1 Introduction 1

1.1 Background 1

1.2 Objectives 1

1.3 Overview 1

2 Methods 2

2.1 Numerical Integration 2

2.2 Numerical Differentiation 2

2.3 Root-Finding Algorithms 2

2.4 Simulation Setup: ODE Solving 2

3 Results 3

3.1 Trajectory Analysis 3

3.2 Data Visualization 3

4 Validation and Error Analysis 6

4.1 Validation Methodology 6

4.2 Error Analysis 6

4.3 Discussion 6

Appendices

A Code Implementations 8

A.1 Main Analysis Script 8

A.2 Data Import 9

A.3 Numerical Methods 10

A.4 Physics Functions 15

A.5 Visualization 18

List of Figures

3.1	Altitude graphed against time, with ODE prediction plot.	4
3.2	Derived velocity plotted against time.	4
3.3	Overall caption of the figure.	5

1 Introduction

1.1 Background

Rocket flight is a straightforward method to study several complex kinematic and aerodynamic concepts. While in-depth spaceflight studies require expensive instruments and technology, the general concepts can be studied using simple position data. The Clemson University Rocket Club designs, builds, and launches medium-sized rockets, and, while they can do comprehensive studies of their builds, student organizations are limited in what data they can collect. The rocket used in this study contains several sensors for collecting data, most importantly an altimeter. An altimeter is used to collect position data, and acceleration data is also collected using an accelerometer. The rocket is a 5-foot-tall rocket with a parachute system that deploys as it falls. Computational methods can be used to complete an extensive analysis of the rocket flight and characterize its path.

1.2 Objectives

The objectives of this study are to fully characterize the flight of the Clemson Rocket Club's rocket by using the rocket's position data from the altimeter to determine the maximum height, total distance traveled, impact of air resistance and drag, and other aspects of the path. Acceleration data will be used in error analysis to determine the validity of differential techniques. This will be done using several computational analysis methods in Matlab, such as interpolation, numerical differentiation and integration, root-finding techniques, and ODE solving. From these data, any flaws in the rocket flight will be identified for recommendation to the Rocket Club for further modification.

1.3 Overview

This report will detail each computational method used to analyze the flight path. It will then discuss the numerical results of the analysis and their physical mean. Error analysis will be discussed to determine the accuracy and truncation error of the calculations. Finally, conclusions and recommendations will be made surrounding the efficiency of the rocket flight.

2 Methods

2.1 Numerical Integration

A numerical integral analysis using the trapezoidal method was used to determine the total distance traveled by the rocket during its flight path. This method considers both the ascent and descent of the rocket, including while the parachute is deployed.

2.2 Numerical Differentiation

Numerical differentiation, in the form of central differences, was used to derive the velocity and acceleration trends from the data. A cubic spline interpolation method was used to smooth the data before deriving. As well as a secondary gaussian filter, this gaussian filter was necessary as the raw accelerometer data has a lot of high frequency noise which resulted in sporadic acceleration values. This approach was used to minimize noise amplification and increase accuracy, instead of using forward and backward differences.

2.3 Root-Finding Algorithms

Important moments in the flight path, such as the apogee, were identified using root-finding methods. The apogee was determined by identifying the point with the maximum altimeter reading. Meanwhile, the velocity extrema were identified at the root of the acceleration trend. The terminal velocity was also determined by finding the root of the position trend and calculating the velocity at that point in time.

2.4 Simulation Setup: ODE Solving

The rocketry data were processed using several numerical methods in order to establish smooth trends for the velocity and acceleration data. The kinematic equation, including air resistance, was solved using this data to determine the drag coefficient of the rocket. Several values of k were trialed with the most physically realistic option being selected for modeling purposes. We do this by minimizing the RMSE between the model and the real data.

3 Results

3.1 Trajectory Analysis

Using the methods detailed above, the analysis of the rocket trajectory reveals an apogee altitude of approximately 570 meters, achieved at a time of nearly 15 seconds. The parachute was deployed approximately 19 seconds prior, as indicated by the deviation in position, velocity, and acceleration trends before that moment. The maximum velocity occurred at 6 seconds after launch and was around 115 m/s. Impact with the ground occurs at 29 seconds at a velocity of -45 m/s. The maximum acceleration occurred at 19 seconds and was -155 m/s^2 . The optimized drag coefficient of k was determined to be 0.0005 kg/s . Visual analysis of the position, velocity, and acceleration profiles confirms the physical accuracy of the path, with the peak of the rocket flight showing the root of the acceleration curve.

3.2 Data Visualization

The figures below show the trajectory of the rocket. Figure 3.1 shows the measured altimeter data after it is smoothed using a cubic spline. This visually represents the ascent and descent of the rocket. In this figure, the modeled data from the ODE is also compared. Figure 3.2 displays the path of the rocket. From this, the maximum and minimum speeds can be identified. The point around 15 seconds, where the trend crosses the horizontal axis, shows the peak of the path, where the rocket switches directions. Figure 3.3 displays the acceleration trends in Figure 3.3a, along with a comparison of that trend to the measured values from the accelerometer. This data contained a significant amount of noise, which is expected for data that has been derived twice. The difference between the measured and derived acceleration data is seen in Figure 3.3b. In each figure, the rocket launch, peak, parachute launch, and impact into the ground can be visualized.

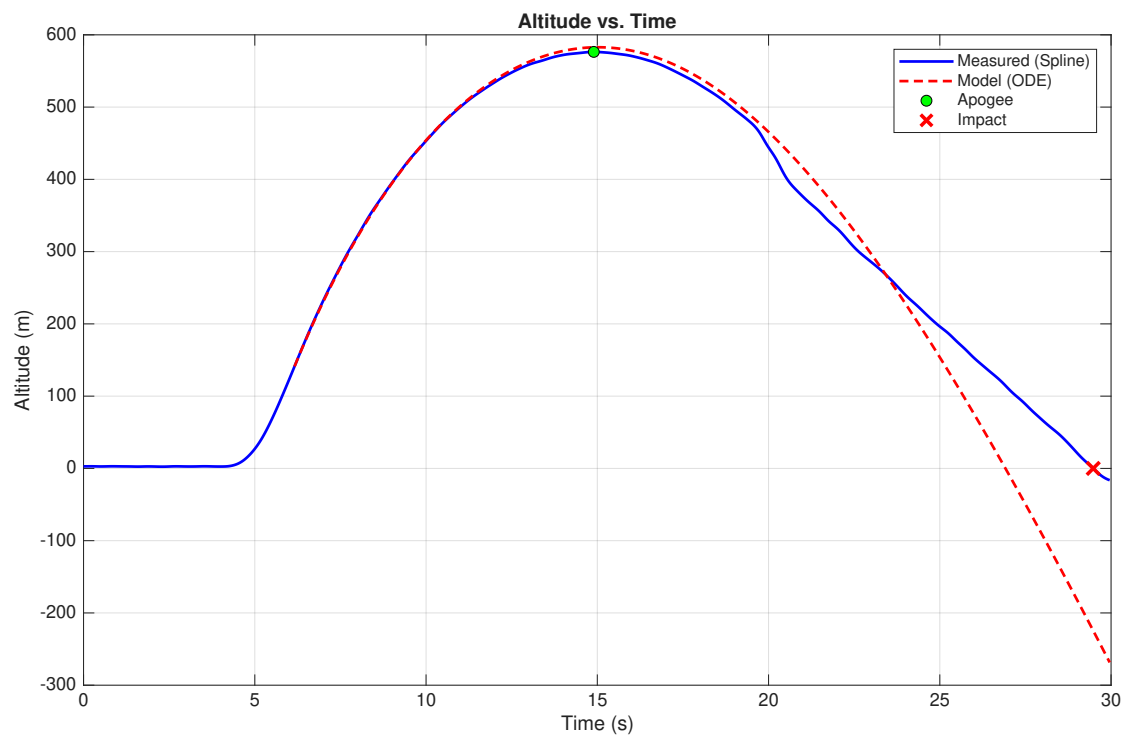


Figure 3.1: Altitude graphed against time, with ODE prediction plot.

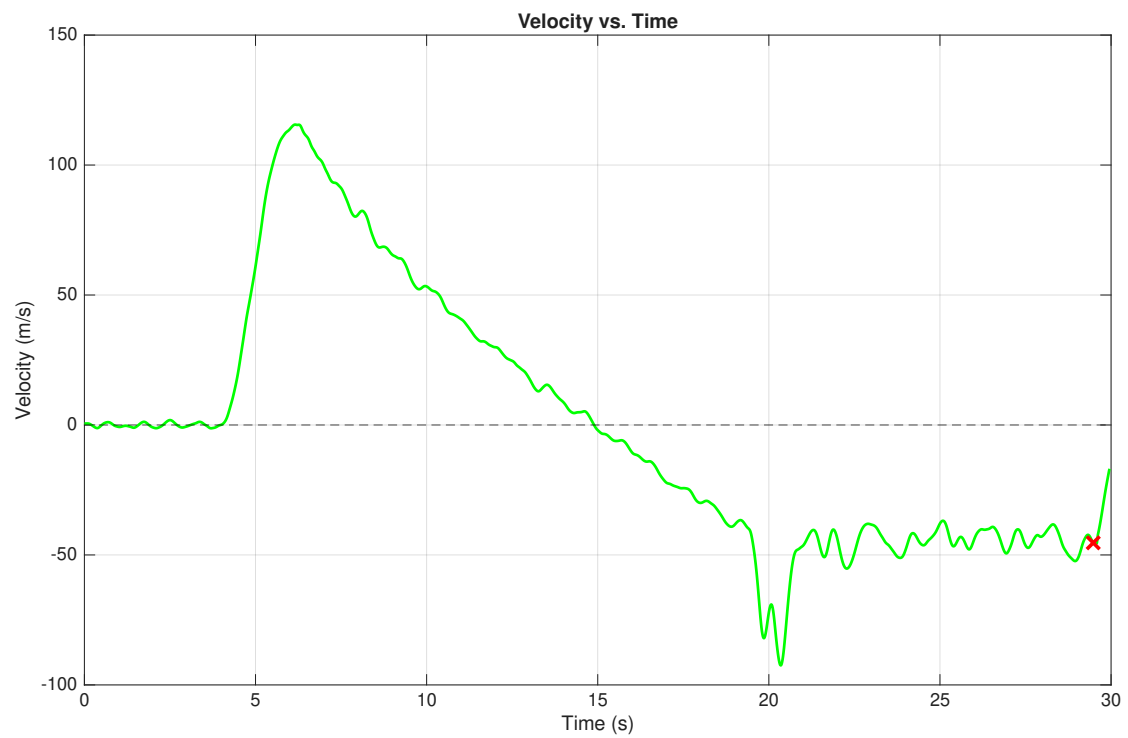
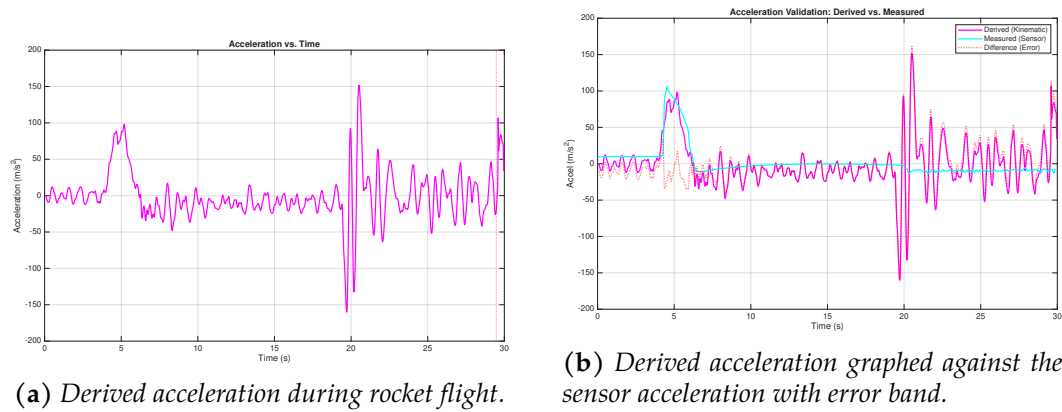


Figure 3.2: Derived velocity plotted against time.



(a) Derived acceleration during rocket flight.

(b) Derived acceleration graphed against the sensor acceleration with error band.

Figure 3.3: Overall caption of the figure.

4 Validation and Error Analysis

4.1 Validation Methodology

The trends of position, velocity, and acceleration of the path were graphically visualized and verified to be reasonable. The figures show a velocity increase as the rocket launches, a decrease as it reaches the apogee, and then a negative increase as it descends. The acceleration trend depicts the acceleration decreasing after the initial launch until it reaches zero at the apogee, and then it also begins increasing negatively.

4.2 Error Analysis

The data analyzed was taken from an altimeter sensor on the rocket. That data was used in several computational methods to characterize the rocket trajectory. This data, specifically the derived acceleration trend, was compared to data taken from the accelerometer taken from the rocket. This error can be seen in Figure 3.3b. The noise in the data can be seen in the oscillations of the acceleration trends

4.3 Discussion

Overall, the rocket trajectory indicates that Clemson's Rocket Club has an aerodynamic rocket that is capable of exhibiting a traditional trajectory. To optimize the rocket, they should consider making modifications to decrease the drag coefficient. Another recommendation would be to study the time at which the parachute is deployed. The magnitude of the terminal velocity suggests that repeated testing at this speed may cause future damage to the rocket. Adjusting the time when the parachute is deployed or potentially investigating a larger parachute will allow for a softer, less intense impact. Potential modifications that could assist the club in collecting better data for analysis would be to test the altimeter at different locations along the rocket to determine which gives the most accurate, stable data. Some of the data noise may result from vibrations of the rocket or bumps from the air. It is possible that certain areas of the rocket may be less susceptible to vibration, resulting in smoother and more accurate data. Additionally, the accelerometer was found to be significantly more precise than the altimeter. Therefore, upgrading the altimeter to match the data collection capabilities of the accelerometer would enable the collection of more precise data for analysis.

Appendices

A Code Implementations

This appendix contains the MATLAB code implementations used for the rocket flight analysis presented in this report. The code is organized into five functional categories: the main orchestration script, data import utilities, numerical methods, physics-specific functions, and visualization tools.

A.1 Main Analysis Script

Listing 1: Main rocket analysis orchestration script

```
1  % Rocket Flight Analysis - by Alex Korte & Julia Jacques
2  %
3  % This script analyzes rocket flight altimeter data to estimate a whole host of
   ↪ parameters.
4  % It performs data import, interpolation, differentiation, root finding,
5  % integration, and ODE modeling.
6
7  clear; close all; clc;
8
9  % Import all the other modules (had to make it multiple folders cause it was becoming a
   ↪ mess)
10 addpath('src');
11 addpath('src/data');
12 addpath('src/numerical');
13 addpath('src/physics');
14 addpath('src/plotting');
15
16 %% 1. Setup and Constants
17 filename = 'ROCKET-FLIGHT-DATA.csv';
18 g = 9.81;      % Gravity (m/s^2)
19 mass = 0.5;    % Mass (kg) - This is kinda a rough estimate because the launch was so
   ↪ long ago
20
21 fprintf('Running Rocket Flight Analysis...\n');
22
23 %% 2. Data Import
24 fprintf('Importing data...\n');
25 [time_raw, alt_raw, accel_raw] = importData(filename);
26
27
28 %% 3. Interpolation
29 fprintf('Interpolating altitude data...\n');
```

```

30 % Create a uniform high-res time vector for analysis
31 num_points = 2000;
32 [time, alt_spline] = interpolateAltitude(time_raw, alt_raw, num_points);
33
34 % Interpolate ground truth acceleration data for validation of our derivations
35 accel_interp = interp1(time_raw, accel_raw, time, 'linear');
36
37 % Use the spline interpolation as it helps prevent the differentiation from
38 % being too noisy, also apply a gaussian filter to remove some of the high frequency
39 % sensor noise
40 alt_analysis = smoothdata(alt_spline, 'gaussian', 50);
41
42 %% 4. Numerical Differentiation
43 fprintf('Calculating velocity and acceleration...\n');
44 [velocity, acceleration] = numericalDifferentiate(time, alt_analysis);
45
46 %% 5. Root Finding
47 fprintf('Finding key flight events...\n');
48 [apogee_time, apogee_height, max_vel_time, max_vel_val, max_acc_time, max_acc_val,
49 % impact_time, impact_vel] = ...
50 % findRoots(time, alt_analysis, velocity, acceleration);
51
52 fprintf(' Apogee:           %.2f m at %.2f s\n', apogee_height, apogee_time);
53 fprintf(' Max Velocity:      %.2f m/s at %.2f s\n', max_vel_val, max_vel_time);
54 fprintf(' Max Acceleration: %.2f m/s^2 at %.2f s\n', max_acc_val, max_acc_time);
55 fprintf(' Impact:           %.2f m/s at %.2f s\n', impact_vel, impact_time);
56
57 %% 6. Numerical Integration
58 fprintf('Calculating total distance traveled...\n');
59 total_dist = numericalIntegrate(time, velocity);
60 fprintf(' Total Distance:   %.2f m\n', total_dist);
61
62 %% 7. ODE Solving (Drag Estimation)
63 fprintf('Estimating drag coefficient...\n');
64 [k_est, t_model, y_model] = solveDragODE(time, alt_analysis, velocity, mass, g,
65 % max_vel_time, apogee_time);
66 fprintf(' Estimated k:      %.4f kg/s\n', k_est);
67
68 %% 8. Visualization
69 fprintf('Plotting results...\n');
70 % Find index of apogee for plotting
71 [~, apogee_idx] = min(abs(time - apogee_time));
72 plotResults(time, alt_analysis, velocity, acceleration, t_model, y_model, apogee_idx,
73 % accel_interp, impact_time, impact_vel);
74
75 fprintf('Analysis Complete.\n');

```

A.2 Data Import

Listing 2: Data import and preprocessing function

```

1 function [time, altitude, accel_vertical] = importData(filename)
2 % IMPORTDATA Import and preprocess rocket flight data
3 %   [time, altitude, accelz] = importData(filename)
4
5 data = readtable(filename);
6
7 % Extract columns
8 % Extract columns
9 raw_time = data.time;
10 altitude = data.altitude;
11 accel_vertical = data.accelx; % accelx is vertical (approx 9.8 on pad)
12
13 % Normalize time (assuming nanoseconds based on magnitude)
14 % Shift to start at 0 and convert to seconds
15 time = (raw_time - raw_time(1)) / 1e9;
16
17 % Ensure data is sorted by time
18 [time, sortIdx] = sort(time);
19 altitude = altitude(sortIdx);
20 accel_vertical = accel_vertical(sortIdx);
21 end

```

A.3 Numerical Methods

These are all the numerical methods we learned in class.

Listing 3: Bisection method for root finding

```

1 function [root, iter] = bisection(f, a, b, tol, maxIter)
2 % BISECTION Root-finding via bisection method
3 %   [root, iter] = bisection(f, a, b, tol, maxIter)
4
5 % initial guess
6 an = a;
7 bn = b;
8 fa = f(an);
9 fb = f(bn);
10 if fa*fb > 0
11     error('No sign change in interval [%f, %f].', an, bn);
12 end
13
14 % iteration to narrow down the interval
15 for iter = 1:maxIter
16     % evaluate the midpoint
17     m = (an + bn)/2;
18     fm = f(m);
19
20     if abs(fm) < tol || (bn-an) < tol
21         % convergence condition
22         break;

```

```

23     end
24     if fa*fm < 0
25         % choose the left half
26         bn = m;
27         fb = fm;
28     else
29         % choose the right half
30         an = m;
31         fa = fm;
32     end
33 end
34
35 root = (an+bn)/2;
36 end

```

Listing 4: Natural cubic spline interpolation implementation

```

1  function yq = cubicSplineInterp(x, y, xq)
2  %CUBICSPLINEINTERP Natural cubic spline interpolation
3  %
4  %   yq = cubicSplineInterp(x, y, xq)
5  %
6  % Inputs:
7  %   x - data point positions (must be strictly increasing)
8  %   y - data point values
9  %   xq - query points (scalar or vector)
10 %
11 % Output:
12 %   yq - interpolated values at xq
13
14 n = numel(x);
15 if numel(y) ~= n
16     error('x and y must have the same length.');

```

```

36     A(i,i-1) = h(i-1);
37     A(i,i)   = 2*(h(i-1)+h(i));
38     A(i,i+1) = h(i);
39     rhs(i)   = 6*((y(i+1)-y(i))/h(i) - (y(i)-y(i-1))/h(i-1)));
40 end
41
42 % Solve system
43 M = A\rhs;
44
45 % Evaluate spline at query points
46 yq = zeros(size(xq));
47 for k = 1:numel(xq)
48     % Find interval containing xq(k)
49     i = find(xq(k) >= x(1:end-1) & xq(k) <= x(2:end), 1);
50     if isempty(i)
51         % Extrapolation: clamp to nearest interval
52         if xq(k) < x(1)
53             i = 1;
54         else
55             i = n-1;
56         end
57     end
58     hi = x(i+1) - x(i);
59
60     % Cubic spline formula
61     term1 = M(i) * (x(i+1)-xq(k))^3 / (6*hi);
62     term2 = M(i+1) * (xq(k)-x(i))^3 / (6*hi);
63     term3 = (y(i) - M(i)*hi^2/6) * (x(i+1)-xq(k))/hi;
64     term4 = (y(i+1) - M(i+1)*hi^2/6) * (xq(k)-x(i))/hi;
65
66     yq(k) = term1 + term2 + term3 + term4;
67 end
68 end

```

Listing 5: Modified Euler method (Heun's method) for ODE solving

```

1 function [t, y] = euler_modified(f, t_span, y0, h)
2 % EULER_MODIFIED Solve ODE using Modified Euler method (Heun's method)
3 % [t, y] = euler_modified(f, t_span, y0, h)
4 %
5 % f      : function handle for dy/dt = f(t, y)
6 % t_span : [t_start, t_end]
7 % y0     : initial condition
8 % h      : step size
9
10 t = t_span(1):h:t_span(2);
11 y = zeros(length(t), length(y0));
12 y(1, :) = y0;
13
14 for i = 1:length(t)-1
15     % Predictor step (Euler)
16     k1 = f(t(i), y(i, :));

```

```

17     y_predict = y(i, :) + h * k1';
18
19     % Corrector step
20     k2 = f(t(i+1), y_predict);
21     y(i+1, :) = y(i, :) + (h/2) * (k1' + k2');
22 end
23 end

```

Listing 6: Centered difference numerical differentiation

```

1 function dydx = my_diff_ctr(x, y)
2 % MY_DIFF_CTR return the derivative for the function at x with a centered estimation.
3 %
4 %   y : the y values of the function we want to differentiate
5 %   x : the x values of the function we want to differentiate
6
7 % Pretty sure I'm using this right:
8 %   ↪ https://www.mathworks.com/help/matlab/ref/circshift.html
9
10 dydx = (circshift(y, 1) - circshift(y, -1)) ./ (circshift(x, 1) - circshift(x, -1));
11
12 % Use forward difference for the first point and backward difference for the last point
13 dydx(1) = (y(2) - y(1)) / (x(2) - x(1));
14 dydx(end) = (y(end) - y(end-1)) / (x(end) - x(end-1));
15 end

```

Listing 7: Fourth-order Runge-Kutta ODE solver

```

1 function [t_out, y_out, v_out] = rungeKutta4(t_span, y0, v0, k, mass, g, h)
2 % RUNGEKUTTA4 Solve drag ODE using 4th order Runge-Kutta method
3 %   [t_out, y_out, v_out] = rungeKutta4(t_span, y0, v0, k, mass, g, h)
4 %
5 %   t_span : [t_start, t_end] time interval
6 %   y0      : initial altitude
7 %   v0      : initial velocity
8 %   k       : drag coefficient
9 %   mass    : rocket mass
10 %   g       : gravitational acceleration
11 %   h       : step size
12 %
13 %   t_out   : time vector
14 %   y_out   : altitude vector
15 %   v_out   : velocity vector
16 %
17 %   ODEs:
18 %       y' = v
19 %       v' = -g - (k/mass)*v*|v|
20
21 ta = t_span(1);
22 tb = t_span(2);
23
24 % Number of points

```

```

25 npts = ceil((tb - ta) / h) + 1;
26 h = (tb - ta) / (npts - 1);
27
28 % Time grid
29 tt = linspace(ta, tb, npts);
30
31 % Initialize solution arrays
32 y_rk = zeros(1, npts);
33 v_rk = zeros(1, npts);
34
35 % Set initial conditions
36 y_rk(1) = y0;
37 v_rk(1) = v0;
38
39 % Drag coefficient ratio
40 k2m = k / mass;
41
42 % 4th order Runge-Kutta integration loop
43 for i = 1:npts-1
44     % Current state
45     y_i = y_rk(i);
46     v_i = v_rk(i);
47
48     % k1 = f(t_n, Y_n)
49     yp_k1 = v_i;
50     vp_k1 = -g - k2m * v_i * abs(v_i);
51
52     % k2 = f(t_n + h/2, Y_n + k1*h/2)
53     yp_k2 = v_i + vp_k1 * h/2;
54     vp_k2 = -g - k2m * (v_i + vp_k1 * h/2) * abs(v_i + vp_k1 * h/2);
55
56     % k3 = f(t_n + h/2, Y_n + k2*h/2)
57     yp_k3 = v_i + vp_k2 * h/2;
58     vp_k3 = -g - k2m * (v_i + vp_k2 * h/2) * abs(v_i + vp_k2 * h/2);
59
60     % k4 = f(t_n + h, Y_n + k3*h)
61     yp_k4 = v_i + vp_k3 * h;
62     vp_k4 = -g - k2m * (v_i + vp_k3 * h) * abs(v_i + vp_k3 * h);
63
64     % Update: Y_{n+1} = Y_n + h*(k1/6 + k2/3 + k3/3 + k4/6)
65     y_rk(i+1) = y_i + h * (yp_k1/6 + yp_k2/3 + yp_k3/3 + yp_k4/6);
66     v_rk(i+1) = v_i + h * (vp_k1/6 + vp_k2/3 + vp_k3/3 + vp_k4/6);
67 end
68
69 % Return as column vectors to match typical ODE solver output
70 t_out = tt';
71 y_out = y_rk';
72 v_out = v_rk';
73
74 end

```

Listing 8: Trapezoid rule numerical integration

```

1 function integral = trapezoid(f, a, b, n)
2 % TRAPEZOID return the integral using the trapezoid integration formula
3 %
4 % NOTE: Assumes uniform spacing in x
5 %
6 % f : function we want to integrate
7 % a : lower bound of integration
8 % b : upper bound of integration
9 % n : number of intervals
10
11 dx = (b - a) / n;
12
13 integral = 0;
14 for i = 1:n
15     x_left = a + (i - 1) * dx;
16     x_right = a + i * dx;
17     integral = integral + 0.5 * (f(x_left) + f(x_right)) * dx;
18 end
19 end

```

A.4 Physics Functions

These are the functions specifically for the rocket flight analysis.

Listing 9: Key flight event detection (apogee, max velocity, max acceleration, impact)

```

1 function [apogee_time, apogee_height, max_vel_time, max_vel_val, max_acc_time,
↪ max_acc_val, impact_time, impact_vel] = findRoots(time, altitude, velocity,
↪ acceleration)
2 % FINDROOTS Find key flight events (apogee, max velocity, max acceleration, impact)
3 % [apogee_time, apogee_height, max_vel_time, max_vel_val, max_acc_time, max_acc_val,
↪ impact_time, impact_vel] = findRoots(time, altitude, velocity, acceleration)
4 %
5
6 % Find the apogee which is just the maximum height (also when velocity = 0)
7 [apogee_height, idx_apogee] = max(altitude);
8 apogee_time = time(idx_apogee);
9
10 % Find the maximum velocity
11 [max_vel_val, idx_vel] = max(velocity);
12 max_vel_time = time(idx_vel);
13
14 % Find the maximum acceleration
15 [max_acc_val, idx_acc] = max(acceleration);
16 max_acc_time = time(idx_acc);
17
18 % Find where the rocket impacted the ground
19
20 % Clip to search only after apogee occurs, cause everything before that is ascent
21 alt_descent = altitude(idx_apogee:end);

```

```

22
23 % Whenever the altitude goes below 0, we have impact
24 idx_impact_local = find(alt_descent <= 0, 1);
25
26 if isempty(idx_impact_local)
27     % If it never crosses 0 (sensors get shredded sometimes or rocket blows up), take
28     % ↪ the last point
29     impact_time = time(end);
30     impact_vel = velocity(end);
31 else
32     % Adjust index to global time vector
33     idx_impact = idx_apogee + idx_impact_local - 1;
34
35     t1 = time(idx_impact - 1);
36     t2 = time(idx_impact);
37     y1 = altitude(idx_impact - 1);
38     y2 = altitude(idx_impact);
39
40     impact_time = t1 - y1 * (t2 - t1) / (y2 - y1);
41
42     % Interpolate velocity at the exact time of impact so we can get it super precise
43     impact_vel = interp1(time, velocity, impact_time);
44 end

```

Listing 10: Altitude data interpolation using cubic splines

```

1 function [time_interp, alt_spline] = interpolateAltitude(time, altitude, num_points)
2 % INTERPOLATEALTITUDE Generate smooth altitude data using interpolation
3 % [time_interp, alt_spline] = interpolateAltitude(time, altitude, num_points)
4 %
5 % time : original time vector
6 % altitude : original altitude vector
7 % num_points : number of points for the interpolated vectors
8 %
9 % time_interp : interpolated time vector
10 % alt_spline : cubic spline interpolated altitude
11
12 time_interp = linspace(min(time), max(time), num_points);
13 alt_spline = cubicSplineInterp(time, altitude, time_interp);
14 end

```

Listing 11: Velocity and acceleration calculation via numerical differentiation

```

1 function [velocity, acceleration] = numericalDifferentiate(time, position)
2 % NUMERICALDIFFERENTIATE Calculate velocity and acceleration
3 % [velocity, acceleration] = numericalDifferentiate(time, position)
4 %
5 % time : time vector
6 % position : position vector
7 %
8 % velocity : calculated velocity (m/s)

```

```

9  % acceleration : calculated acceleration (m/s^2)
10
11 % Calculate Velocity and Acceleration by taking the first and second derivatives
12 velocity = my_diff_ctr(time, position);
13 acceleration = my_diff_ctr(time, velocity);
14 end

```

Listing 12: Total distance traveled calculation via numerical integration

```

1  function total_distance = numericalIntegrate(time, velocity)
2  % NUMERICALINTEGRATE Calculate total distance traveled by integrating velocity with the
   ↪ trapezoid rule
3  % total_distance = numericalIntegrate(time, velocity)
4  %
5  % time      : time vector
6  % velocity  : velocity vector
7  %
8  % total_distance : integral of velocity
9
10 % We want total distance, so we integrate absolute velocity (speed)
11 speed = abs(velocity);
12 total_distance = trapz(time, speed);
13 end

```

Listing 13: Drag coefficient estimation using ODE modeling with RK4

```

1  function [k_est, t_model, y_model] = solveDragODE(time, altitude, velocity, mass, g,
   ↪ t_start, t_end)
2  % SOLVEDRAGODE Estimate drag coefficient and model trajectory using RK4
3  % [k_est, t_model, y_model] = solveDragODE(time, altitude, velocity, mass, g, t_start,
   ↪ t_end)
4  %
5  % time      : time vector from flight data
6  % altitude  : altitude vector from flight data
7  % velocity  : velocity vector from flight data
8  % mass      : rocket mass (kg)
9  % g         : gravity (m/s^2)
10 % t_start   : start time for fitting (burnout)
11 % t_end     : end time for fitting (apogee)
12 %
13 % k_est     : estimated drag coefficient
14 % t_model   : modeled time vector
15 % y_model   : modeled altitude vector
16 %
17 % Model:  $m \cdot v' = -mg - k \cdot v \cdot |v|$  (Quadratic Drag)
18 % Only fit the coast phase since the parachute deployment messes
19 % up the model, cause its drag characteristics are completely different.
20
21 % Find indices corresponding to t_start and t_end
22 idx_start = find(time >= t_start, 1);
23 idx_end = find(time >= t_end, 1);

```

```

24
25     if isempty(idx_end)
26         idx_end = length(time);
27     end
28
29     % Get only the coast phase data
30     time_fit = time(idx_start:idx_end);
31     alt_fit = altitude(idx_start:idx_end);
32     vel_fit = velocity(idx_start:idx_end);
33
34     % Initial conditions
35     y0 = alt_fit(1);
36     v0 = vel_fit(1);
37
38     t_span_model = [min(time_fit), max(time)];
39
40     % Find optimal k by doing bisection
41     cost_fun = @(k_val) calculateError(k_val, time_fit, y0, v0, alt_fit, mass, g);
42     k_est = fminbnd(cost_fun, 0, 0.1);
43
44     % Generate full trajectory model with best k using 4th order Runge-Kutta
45     h = 0.01;
46     [t_model, y_model, ~] = rungeKutta4(t_span_model, y0, v0, k_est, mass, g, h);
47
48     end
49
50
51     function err = calculateError(k_val, time_fit, y0, v0, alt_fit, mass, g)
52         % CALCULATEERROR Compute RMSE between model and data for a given k
53         % err = calculateError(k_val, time_fit, y0, v0, alt_fit, mass, g)
54
55         h = 0.01;
56         t_span = [time_fit(1), time_fit(end)];
57
58         [t_sol, y_sol, ~] = rungeKutta4(t_span, y0, v0, k_val, mass, g, h);
59
60         % Handle solver failures
61         if length(t_sol) < 2
62             err = 1e9;
63             return;
64         end
65
66         % Interpolate solution to match data points
67         y_sol_interp = interp1(t_sol, y_sol, time_fit, 'linear', 'extrap');
68         err = sqrt(mean((y_sol_interp - alt_fit).^2));
69     end

```

A.5 Visualization

Listing 14: Results visualization and plotting function

```

1 function plotResults(time, altitude, velocity, acceleration, t_model, y_model,
    ↪ apogee_idx, accel_measured, impact_time, impact_vel)
2 % PLOTRESULTS Visualize rocket flight data
3
4 f = figure('Name', 'Rocket Flight Analysis', 'NumberTitle', 'off', 'Position', [100,
    ↪ 100, 1500, 600]);
5
6 tabgrp = uitabgroup(f);
7
8 %% Tab 1: Flight Analysis (Combined)
9 tab1 = uitab(tabgrp, 'Title', 'Flight Analysis');
10 axes('Parent', tab1);
11
12 % 1. Altitude
13 subplot(1, 3, 1, 'Parent', tab1);
14 plot(time, altitude, 'b-', 'LineWidth', 1.5); hold on;
15 plot(t_model, y_model, 'r--', 'LineWidth', 1.5);
16 plot(time(apogee_idx), altitude(apogee_idx), 'ko', 'MarkerFaceColor', 'g');
17 plot(impact_time, 0, 'rx', 'MarkerSize', 10, 'LineWidth', 2);
18 title('Altitude vs. Time');
19 xlabel('Time (s)'); ylabel('Altitude (m)');
20 legend('Measured (Spline)', 'Model (ODE)', 'Apogee', 'Impact');
21 grid on;
22
23 % 2. Velocity
24 subplot(1, 3, 2, 'Parent', tab1);
25 plot(time, velocity, 'g-', 'LineWidth', 1.5); hold on;
26 yline(0, 'k--');
27 plot(impact_time, impact_vel, 'rx', 'MarkerSize', 10, 'LineWidth', 2);
28 title('Velocity vs. Time');
29 xlabel('Time (s)'); ylabel('Velocity (m/s)');
30 grid on;
31
32 % 3. Acceleration
33 subplot(1, 3, 3, 'Parent', tab1);
34 plot(time, acceleration, 'm-', 'LineWidth', 1.5); hold on;
35 xline(impact_time, 'r:', 'LineWidth', 1.5);
36 title('Acceleration vs. Time');
37 xlabel('Time (s)'); ylabel('Acceleration (m/s^2)');
38 grid on;
39
40 %% Tab 2: Altitude (Single)
41 tab2 = uitab(tabgrp, 'Title', 'Altitude');
42 ax2 = axes('Parent', tab2);
43 plot(ax2, time, altitude, 'b-', 'LineWidth', 1.5); hold on;
44 plot(ax2, t_model, y_model, 'r--', 'LineWidth', 1.5);
45 plot(ax2, time(apogee_idx), altitude(apogee_idx), 'ko', 'MarkerFaceColor', 'g');
46 plot(ax2, impact_time, 0, 'rx', 'MarkerSize', 10, 'LineWidth', 2);
47 title(ax2, 'Altitude vs. Time');
48 xlabel(ax2, 'Time (s)'); ylabel(ax2, 'Altitude (m)');
49 legend(ax2, 'Measured (Spline)', 'Model (ODE)', 'Apogee', 'Impact');

```

```

50 grid(ax2, 'on');
51
52 %% Tab 3: Velocity (Single)
53 tab3 = uitab(tabgp, 'Title', 'Velocity');
54 ax3 = axes('Parent', tab3);
55 plot(ax3, time, velocity, 'g-', 'LineWidth', 1.5); hold on;
56 yline(ax3, 0, 'k--');
57 plot(ax3, impact_time, impact_vel, 'rx', 'MarkerSize', 10, 'LineWidth', 2);
58 title(ax3, 'Velocity vs. Time');
59 xlabel(ax3, 'Time (s)'); ylabel(ax3, 'Velocity (m/s)');
60 grid(ax3, 'on');
61
62 %% Tab 4: Acceleration (Single)
63 tab4 = uitab(tabgp, 'Title', 'Acceleration');
64 ax4 = axes('Parent', tab4);
65 plot(ax4, time, acceleration, 'm-', 'LineWidth', 1.5); hold on;
66 xline(ax4, impact_time, 'r:', 'LineWidth', 1.5);
67 title(ax4, 'Acceleration vs. Time');
68 xlabel(ax4, 'Time (s)'); ylabel(ax4, 'Acceleration (m/s^2)');
69 grid(ax4, 'on');
70
71 %% Tab 5: Validation
72 tab5 = uitab(tabgp, 'Title', 'Validation');
73 ax5 = axes('Parent', tab5);
74
75 % Acceleration Comparison
76 plot(ax5, time, acceleration, 'm-', 'LineWidth', 1.5); hold on;
77 plot(ax5, time, accel_measured, 'c-', 'LineWidth', 1.5);
78
79 % Calculate and plot error (Difference)
80 accel_error = acceleration - accel_measured;
81 plot(ax5, time, accel_error, 'r:', 'LineWidth', 1.5);
82
83 title(ax5, 'Acceleration Validation: Derived vs. Measured');
84 xlabel(ax5, 'Time (s)'); ylabel(ax5, 'Accel (m/s^2)');
85 legend(ax5, 'Derived (Kinematic)', 'Measured (Sensor)', 'Difference (Error)');
86 grid(ax5, 'on');
87
88 end

```
